組込みシステムIII

ガイダンス

出席番号順に着席してください
MyIDで出欠を入力してください
Teams コード kgonrax

授業の概要

大規模で高度な機能を持つディジタル回路(論理回路)を設計・実装する力を身に付ける。

設計手法:

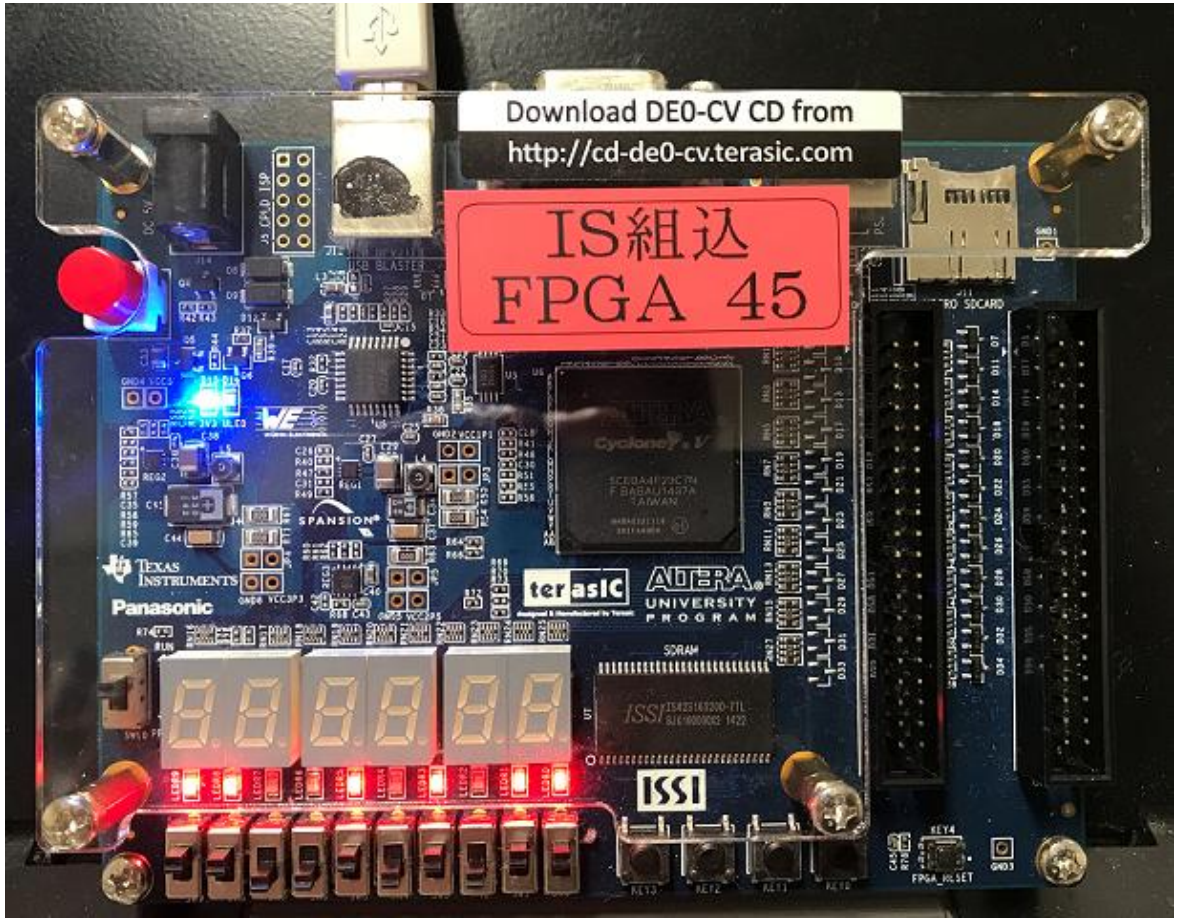
HDL(ハードウェア記述言語)による回路記述

この授業ではHDLとしてSystemVerilogを用いる

シラバス・授業スケジュール・評価

- シラバス・授業スケジュール⇒別紙
- 評価方法：以下を総合的に評価(すべてを提出すること)
  - 小レポート課題(2~3回予定)：
    - ✓ HDLソースコード, 動作例, 検証結果
  - 最終レポート：実習課題に取り組み実験レポートを提出
    - ✓ 実験レポート(目的、原理、手法、結果、考察)
  - 動作確認：実習課題のデモ・口頭試問

実習ボード(FPGA)



DE0-CV ：

FPGAを搭載したディジタル回路の評価ボード

搭載FPGA：

Cyclone V E FPGA(Intel FPGA)

実習では、
HDLで設計した回路情報を
FPGA実習ボードに書き込み
回路の実装と動作確認を行う

FPGAとは

Field Programmable Gate Array :

現場で回路構成を設定できる集積回路 (CPUではない)

特徴 :

- 多くの(数百~数百万)の論理ブロック(論理ゲートやフリップフロップに相当)で構成
- 回路情報を書き込むことで、各論理ブロックの働きやブロック間の接続を切り替え、望むディジタル回路を構築

HDLで設計したディジタル回路をFPGAで実装できる

FPGA vs CPU(マイコン)

CPU:

- プログラムに従って一つ一つ処理を行う(順次処理)
- プログラムの書き換えで容易に処理を変えられる(汎用性)

FPGA(によるディジタル回路):

- 内部に同じ処理を行う回路を複数構築でき、並列して処理を行える(並列処理)⇒高速化・リアルタイム処理が見込める
- 処理を変えるには回路構成を書き換える必要あり
- 回路構成の書き換えが他の集積回路より容易なのがFPGAの利点

FPGAの応用例

回路の書き換えが可能⇒汎用性・柔軟性が高い

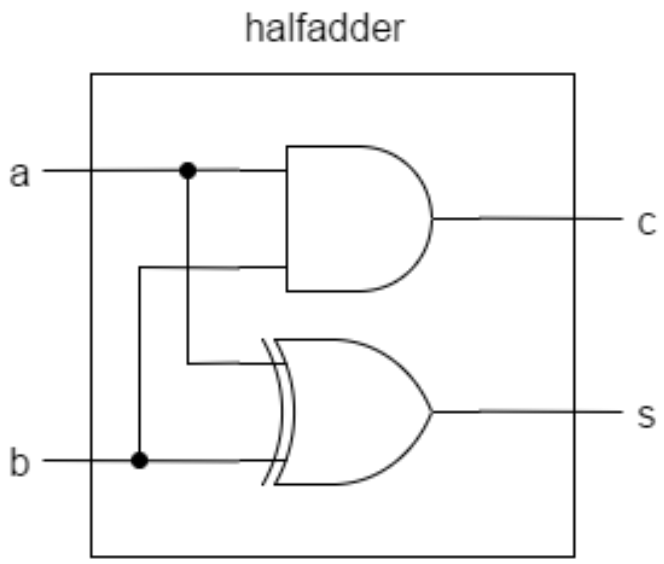
多数の回路で並列処理⇒高速・高効率な処理

- 画像処理
  8K映像のエンコード/デコード、カメラ画像からの物体検出
- 測定器のデータ計測・処理
  HLC(大型ハドロン衝突型加速器)の検出器用の計測ボード
- ネットワークスイッチ
  5G基地局
- 機械学習

HDL(ハードウェア記述言語)

HDL: Hardware Description Language

<u>ディジタル回路(論理回路)の論理的な構成や動作を記述</u>するための(人工)言語




```
module halfadder(  // モジュール定義
  input  logic a, input  logic b, // 入力信号
  output logic c, output logic s  // 出力信号
);

  assign c = a & b;  // モジュール内部の
  assign s = a ^ b;  // 構成(配線・動作)

endmodule
```

処理の流れを書いた**プログラムではない**ことに注意！

HDLによるディジタル回路の開発手順

1. 回路仕様の決定
   - 入出力インターフェース
   - 入出力の関係、状態遷移
2. HDLでの回路設計
3. シミュレーションによる動作検証
   - 論理レベルでのシミュレーション
   - 遅延を考慮したタイミングシミュレーション
4. FPGAへの回路情報書き込み(実装)と動作検証

この授業では3.シミュレーションによる動作検証は扱わない

HDLでの回路設計の注意点
–– プログラミングとの違い––

- HDLでは回路の構成や振る舞いを記述
- 処理の流れを書いたプログラムではない
  - 順次処理、繰り返し処理は記述できない
    ⇒これらの処理を行いたい場合は順序回路を設計する
- 回路図、真理値表、状態遷移図、タイムチャートなどを用いて回路設計を行いHDLへ変換するとよい
  - 処理の流れを記述するフローチャートは回路設計には不向き

ディジタル回路(論理回路)の復習 1

組合せ回路の設計

次の真理値表を満たす組み合わせ論理回路(以下、トグル回路と呼ぶ)を設計せよ。AND, OR, NOT ゲートを用いること。

トグル回路の真理値表

入力		出力
t	x	y
0	0	0
0	1	1
1	0	1
1	1	0

トグル回路をHDL(SystemVerilog)で設計すると

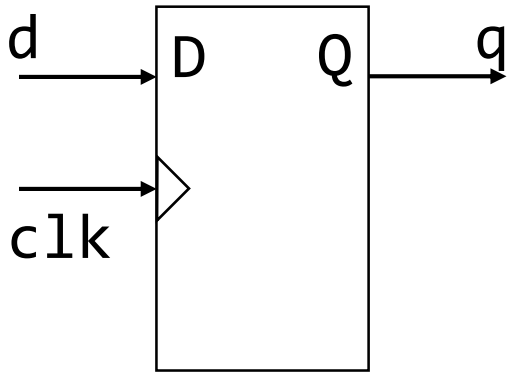
トグル回路の真理値表

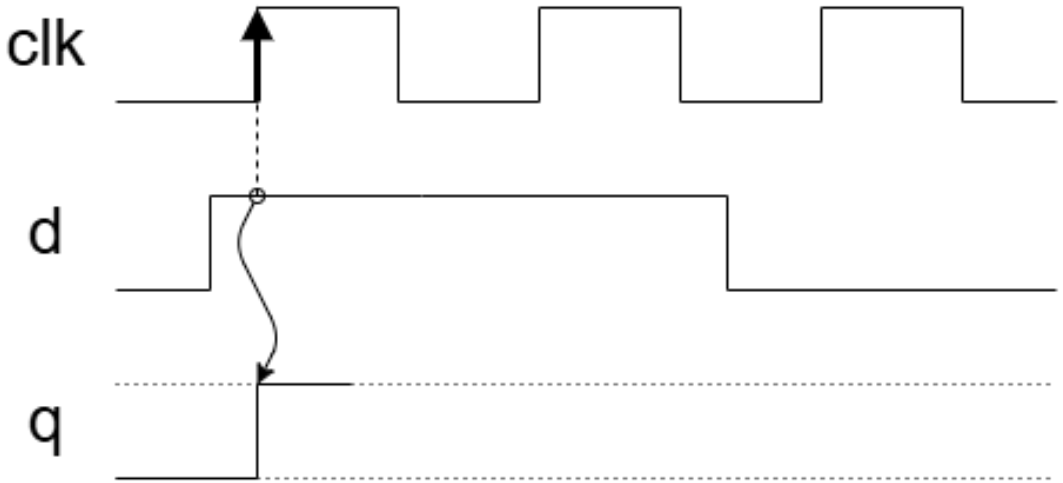
入力		出力
t	x	y
0	0	0
0	1	1
1	0	1
1	1	0

```
module toggle_cirquit(
  input logic t, input logic x,
  output logic y);

  always_comb begin
    case ({t, x})
      2'b00 : y = 1'b0;
      2'b01 : y = 1'b1;
      2'b10 : y = 1'b1;
      2'b11 : y = 1'b0;
    endcase
  end
endmodule
```

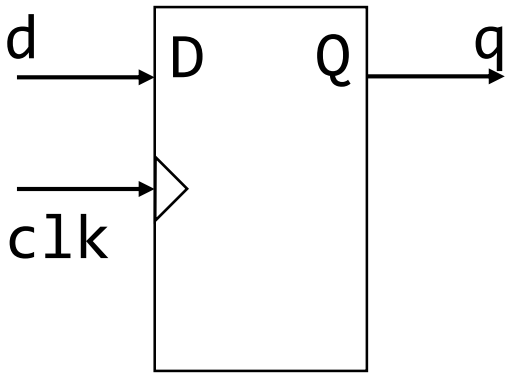
ディジタル回路(論理回路)の復習 2

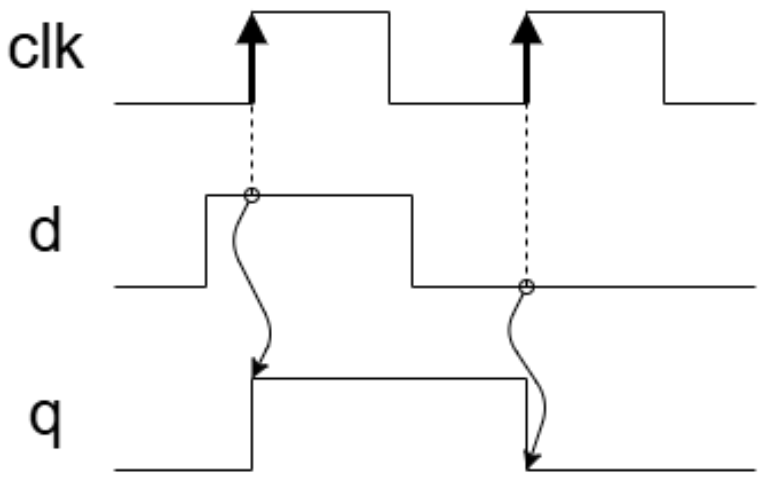
順序回路の設計
D-FF(Dフリップフロップ)のタイムチャートを完成させよ
(クロックの立ち上がりのタイミングで出力が変化することに注意)





D-FFをHDL(SystemVerilog)で設計すると



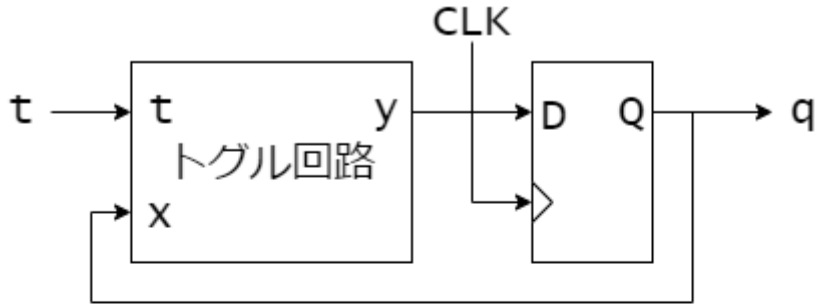


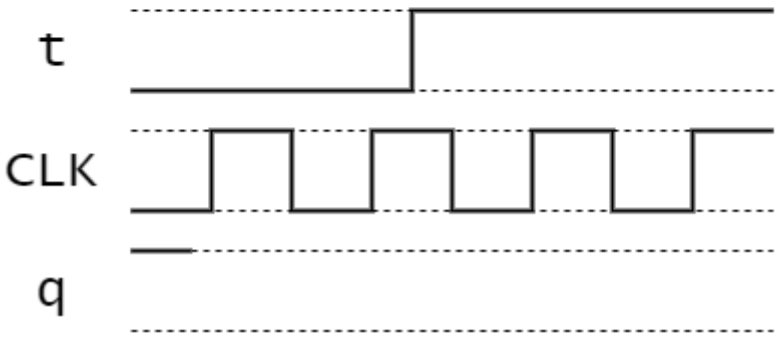

```
module dff(
  input  logic clk,
  input  logic d,
  output logic q);

  always_ff @ (posedge clk) begin
    q <= d;
  end

endmodule
```

ディジタル回路(論理回路)の復習 3

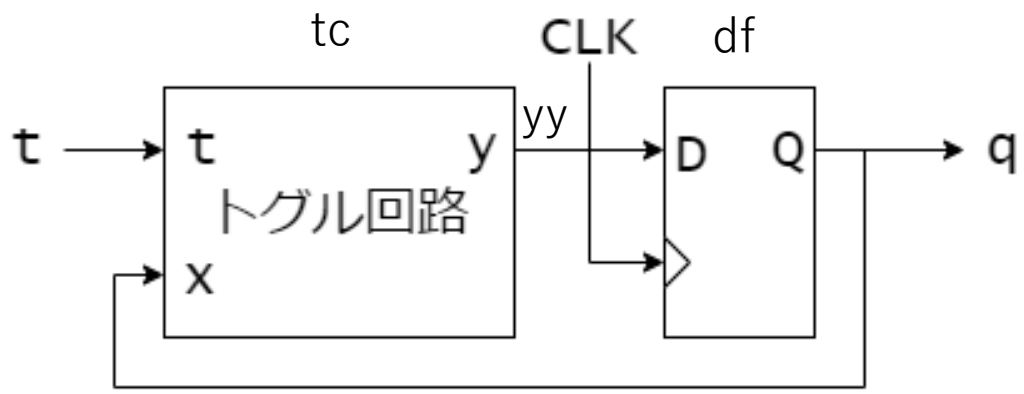
回路モジュールの組み合わせ

トグル回路とD-FFを組み合わせた回路のタイムチャートを完成させよ



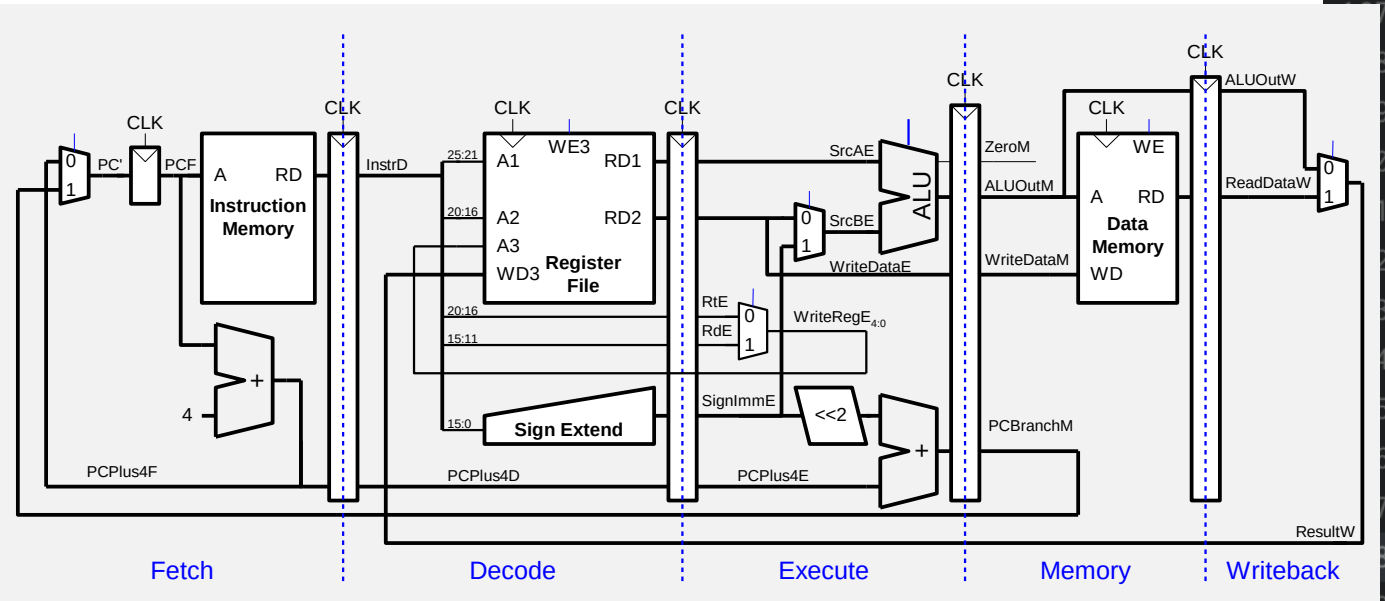


HDL(SystemVerilog)で設計すると




```
module composite_cirquit(
  input  logic clk, input logic t,
  output logic q);

  logic yy;

  toggle_circuit tc(
    .t(t), .x(q), .y(yy));

  dff df(
    .d(yy), .q(q));

endmodule
```

CPUもディジタル回路、頑張ればCPUも作れる！




```
module datapath(input  logic        clk, reset,
                input  logic        memtoreg, pcsrc,
                input  logic        alusrc, regdst,
                input  logic        regwrite, jump,
                input  logic [2:0]  alucontrol,
                output logic        zero,
                output logic [31:0] pc,
                input  logic [31:0] instr,
                output logic [31:0] aluout, writedata,
                input  logic [31:0] readdata);

  logic [4:0]  writereg;
  logic [31:0] pcnext, pcnextbr, pcplus4, pcbranch;
  logic [31:0] signimm, signimmsh;
  logic [31:0] srca, srcb;
  logic [31:0] result;

  // next PC logic
  flopr #(32) pcreg(clk, reset, pcnext, pc);
  adder       pcadd1(pc, 32'b100, pcplus4);
  sl2         immsh(signimm, signimmsh);
  adder       pcadd2(pcplus4, signimmsh, pcbranch);
  mux2 #(32)  pcbrmux(pcplus4, pcbranch, pcsrc, pcnextbr);
```